Exact Posteriors for Values, Exact Costs for Choices: Two-Stage Learned Branching for Binary Linear Systems

Gwang-Jong Ko[a], Taesu Cheong[a,*], In-Chan Choi[a,*]

[a]School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul, 02841, Republic of Korea

Abstract

A binary linear system (BLS) asks for $\mathbf{x} \in \{0,1\}^n$ with $A\mathbf{x} = \mathbf{b}$ for a 0/1 matrix A. The problem has no objective, its linear relaxation carries little information on the instances that arise in practice, and unit propagation is inert near the root of a search tree, so a complete search is decided by its branching choices. We present a branching heuristic for the depth-first search of a BLS that is learned in two stages from two exact signals. Stage 1 learns *which value* to try first: on instances small enough to enumerate every solution, the conditional marginal $\Pr[x_j{=}1 \mid A, \mathbf{b}, \text{partial assignment}]$ is exact, and because conditioning a BLS on a partial assignment is the same as reducing it, a single graph network trained on reduced instances predicts at every node of the tree. Stage 2 learns *which variable* to branch on: the cost of a decision, the number of nodes its subtree expands, is produced exactly by the search, and an actor-critic fine-tuning on that cost, started at the Stage-1 rule with the marginal network frozen, changes only the branching order. The deployed solver runs lattice reduction first and the guided search on what it leaves. On 100 instances of size 21×60 with matched wall-clock budgets and three seeds, Stage 1 expands 5.5× fewer nodes than LP-guided branching and is 2.0–2.3× faster; Stage 2 removes a further 2.3–2.4× of the nodes and 2.0–2.4× of the time ($p < 0.001$), replicates across training seeds, and transfers to sizes it never saw: at 24×70 the fine-tuned policies solve 30/30 instances 2.3–3.3× faster than Stage 1, and at 28×80 they double the solve rate within budget. Ablations show that the two signals are not interchangeable: raising the value accuracy of the marginal network where its margin lives enlarges the tree, while the cost-tuned policy branches on variables the network is *less* sure about and shrinks it.

Keywords: Binary linear systems, Learning to branch, Exact conditional marginals, Reinforcement learning, Tree search, Graph neural networks

1. Introduction

A binary linear system (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0,1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \qquad A \in \{0,1\}^{m\times n},\ \mathbf{b} \in \mathbb{Z}^m_{\geq 0}, \tag{1}$$

with solution set $\mathcal{S}$. It is the general form of a reconstruction task in laser-based non-destructive testing: a binary occupancy image must reproduce a set of integer projections exactly, the scanned object exists, so a solution exists and is often not unique, and the reconstruction has to keep pace with a production line. The instances that matter are dense, with right-hand sides near the middle of each row's range, and they are hard out of proportion to their size.

Three properties defeat the standard tools on such instances. There is no objective, so the bound-based pruning of branch-and-bound has nothing to work with. The linear relaxation is always feasible but carries little information: the relaxation polytope is wide, its vertices lie far from $\{0,1\}^n$, and rounding a vertex predicts a variable's value only slightly better than chance at the root. Unit propagation, the workhorse of constraint solvers, fires only when a row is saturated, which never happens at the root and rarely near it. The classical remedy for this family, lattice basis reduction, solves small instances outright but stops working as the size grows, and complete solvers with clause learning, while fast, do not make the branching decision itself any cheaper to get right. What is left to a complete search is the quality of its branching decisions, and that is the object of this paper.

A branching decision has two parts, and we show that they are best learned from two different exact signals. *Which value* a variable should take first is a question about where the solutions are; on instances small enough to enumerate, the conditional marginal of x_j given the assignments made so far answers it exactly. *Which variable* to branch on is a question about the cost of what follows; the number of nodes the subtree expands answers it exactly, and the search produces that number as it backtracks. Stage 1 of our method trains a bipartite graph network on the first signal. A reduction identity (Proposition 1) makes the same network valid at every node of the tree, and the marginals partition the variables of a node into those whose wrong value empties the subtree (FORCED) and those whose either value still reaches a solution (OPEN); only the former cost a backtrack, and their share grows steeply with depth, which is why the network is trained inside the tree. Stage 2 fine-tunes the branching order on the second signal with the Stage-1 net-

*Corresponding authors.

Email addresses: tcheong@korea.ac.kr (Taesu Cheong), ichoi@korea.ac.kr (In-Chan Choi)

work frozen, so that whatever improves can be attributed to the order alone. The deployed solver (Figure 1) runs the classical lattice stage first and the guided search on what it leaves; the network only guides, so the search stays complete and a wrong prediction costs backtracking and nothing else.

Contributions..

1. A branching heuristic for BLS learned from exact conditional marginals inside a complete search. The reduction identity that carries the target to every node, the FORCED/OPEN partition, and the finding that in-tree supervision is what makes learned guidance better than relaxation rounding (a network trained on root states is worse) are new.
2. A cost-aware fine-tuning of the branching order that starts from the posterior-derived rule and trains only the order. The starting point and the frozen marginal are what let the gain be attributed and what make it transfer to larger instances than it was trained on.
3. Evidence that the two signals capture different things: sharpening the value signal where its margin lives enlarges the tree, while the cost-tuned policy prefers variables the value model is less sure about and shrinks it.
4. A controlled evaluation with solution multiplicity matched across sizes, matched wall-clock budgets, an identical search loop for every branching rule, three evaluation seeds, two training seeds, a 100-instance test set, comparisons against CP-SAT and SCIP, and the deployed cascade with lattice reduction in front.

Section 2 reviews related work and states what it leaves open for this problem, Section 3 states the problem and the reduction identity, Section 4 the method, Section 5 the experiments, organised around four questions, and Section 6 concludes with the limitations and future work.

2. Related work

Marginals of the solution set as a branching signal.. Branching on the fraction of solutions that agree with a decision predates learning: counting-based search (Pesant et al., 2012) branches on the variable–value pair of maximum solution density computed inside individual constraints, and belief-propagation-guided decimation (Montanari et al., 2007) fixes the most biased variable under approximate marginals. NSNet (Li and Si, 2022) trains a graph network on exact marginals of the satisfying assignments of a SAT formula, obtained by enumerating them, and rounds the estimates into an initial assignment for local search; MIP-GNN (Khalil et al., 2022), Neural Diving (Nair et al., 2020) and predict-and-search (Han et al., 2023) predict per-variable biases from collected near-optimal solutions and use them to fix variables, warm-start or restrict a search. Stage 1 shares NSNet's target and differs in where and how it is used: the marginals are conditional and carried to every node of a complete search by the reduction identity, they guide branching without fixing anything, and the FORCED/OPEN partition and the controlled in-tree-versus-root comparison have no counterpart in that work.

Learning to branch.. Khalil et al. (2016) and Gasse et al. (2019) imitate strong branching, the latter with the bipartite variable–constraint graph network we also use; imitation targets an expert's ranking rather than the tree size the ranking is meant to reduce. Optimising tree size directly is the subject of a line of reinforcement-learning work: FMSTS (Etheve et al., 2020) uses the size of a node's subtree as an observable Q-value, shows that under depth-first search minimising every subtree minimises the whole tree, and learns from scratch; tree MDPs (Scavuzzo et al., 2022) formalise the per-subtree credit assignment; retro branching (Parsonson et al., 2023) learns from retrospectively extracted subtree trajectories; SORREL (Feng and Yang, 2025) initialises from demonstrations. Neuro# (Vaezipoor et al., 2021) trains a graph network on the residual formula of a #SAT solver by evolution strategies and generalises to larger instances. Stage 2 uses the FMSTS credit assignment with a plain policy gradient (Williams, 1992); what it adds is the starting point and the scope, a rule derived from exact marginals, frozen, with only the branching order learned. The classical statements of the objective are the fail-first principle (Haralick and Elliott, 1980) and impact-based search (Refalo, 2004). Value ordering has been learned in constraint programming from solutions (Chu and Stuckey, 2015) and by deep Q-learning (Cappart et al., 2021); we take the value from the exact posterior and learn only the variable.

Market split, lattice methods and complete solvers.. The market split problem of Cornuéjols and Dawande (1999) is the benchmark that exposes the weakness of branch-and-bound on systems like (1): m equality rows over $n = 10(m-1)$ binary variables, coefficients drawn uniformly from $\{0, \dots, 99\}$, and each right-hand side set to half its row sum, so that the relaxation polytope is wide, its vertices far from integrality, and bounding prunes almost nothing; branch-and-cut stops at $m \approx 7$. Aardal et al. (2000b,a) solved these instances by reformulating them on a lattice: the Aardal–Hurkens–Lenstra embedding turns a solution into one of the shortest vectors of a lattice, LLL/BKZ reduction (Lenstra et al., 1982; Schnorr and Euchner, 1994) finds it, and the recovered vector is verified, so the method is sound. Lattice enumeration remains the state of the art on the benchmark: Wassermann (2025) solves QOBLIB market-split instances up to $m = 14$ on one CPU. Our instances share the ingredients that make market split hard, dense rows and right-hand sides at the middle of their range, and differ in having $0/1$ coefficients, a planted solution and a search regime of up to 80 variables; lattice reduction is therefore the natural first stage of a BLS solver, and our contribution begins where it stops working. Constraint solvers with clause learning (Marques-Silva and Sakallah, 1999; Ohrimenko et al., 2009) manufacture information as they descend and set the wall-clock reference we compare against. Direct satisfiability prediction has a weak record (Selsam et al., 2019; Li et al., 2023), and message-passing networks provably cannot separate some feasible/infeasible pairs (Chen et al., 2023). Both learned stages of our method are instances of amortised inference (Amos, 2023; Gershman and Goodman, 2014): the marginal network replaces one relaxation per variable, the cost head replaces a completed search.

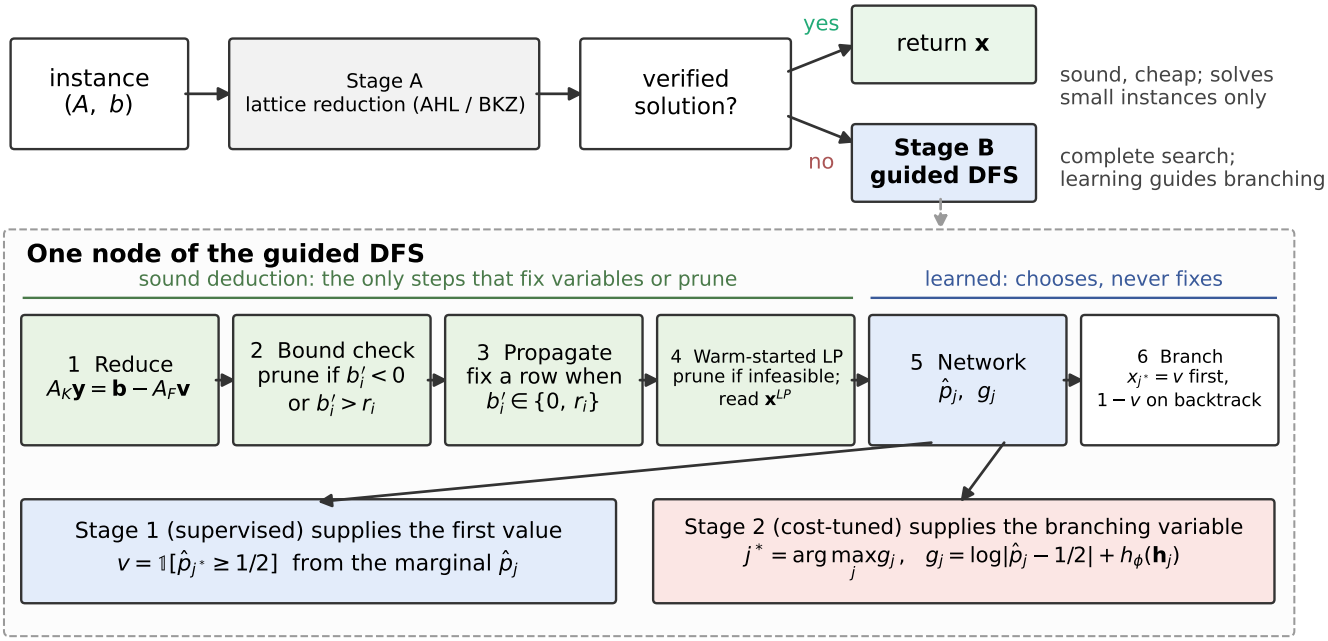


Figure 1: The deployed solver. Stage A, lattice reduction, returns a verified solution when it finds one; Stage B, the guided depth-first search, handles the rest. Inside a node, steps 1–4 are sound and are the only steps that fix variables or prune; the network at step 5 only chooses, the value from Stage 1 and the variable from Stage 2.

What these approaches leave open for a BLS.. The three learned-branching families closest to our setting each lose something essential when transplanted to a feasibility system, and Section 5 tests each transplant inside the same search loop. *Imitation of strong branching* (Khalil et al., 2016; Gasse et al., 2019) presupposes an objective: the expert scores a candidate by the bound improvement of its two children, and a BLS has no bound to improve. The only information strong branching retains on a feasibility system is whether a child's relaxation is infeasible, so the expert degenerates to LP probing, a rule that fixes what it can refute and then branches on the LP value. We run that expert directly; it expands the fewest nodes of any unlearned rule but pays one LP per variable per node, and its wall-clock is 2–3× that of the learned policy at 21×60 and 24×70 and its solve rate at 28×80 is the lowest of the field. Imitating it would inherit its ranking without its cost. *Approximate marginals by message passing* (Li and Si, 2022; Montanari et al., 2007) are well defined for a BLS: each row is a cardinality factor, and sum-product with exact factor messages is implementable. Two things go wrong. The factor graph of a BLS is dense, every row touching about half of the variables, so loopy belief propagation diverges without heavy damping and takes hundreds of iterations where it converges; and NSNet-style use estimates marginals once at the root and decodes an assignment for an incomplete local search, whereas a complete search needs marginals *conditional* on the partial assignment at every node. We run damped sum-product in the Stage-1 slot: on the instances where it finishes, its trees are the size of Stage 1's, but one estimate costs 0.1–1 s against 1 ms for the network forward pass, and within the time budget it solves a quarter of the 21×60 set. The learned marginal network is, in this sense, amortised belief propagation: the same value signal at a thousandth of the price. *Reinforcement learning from scratch on subtree size* (Etheve et al., 2020; Scavuzzo et al., 2022) supplies the right credit assignment, and we keep it. What it does not supply is the value choice: a single policy over variable–value pairs must learn which value to try first from the same sparse reward that teaches it which variable to branch on, while a BLS makes the optimal value available exactly, as a conditional marginal, on the instances it can be enumerated on. Separating the two signals is what the cross ablation in Section 5.5 measures: the cost-tuned variable choice with the LP value in place of the marginal value expands 1.5× more nodes. The initialisation of the policy head, by contrast, turns out not to matter: a head started at random, at the LP rule or at the marginal rule reaches the same tree size once the trunk is shared and frozen, and we report this as a negative result rather than a design choice. Two further gaps are not about learning. Lattice reduction with bounded enumeration (Aardal et al., 2000a; Wassermann, 2025) is sound, and on our evaluation sizes it is the fastest method we know of; it is a preprocessing step that any BLS solver should run first, and the comparison of branching rules in this paper is therefore made on the search that follows it, with the lattice stage switched off. And a network cannot be trusted with a verdict on feasibility (Selsam et al., 2019; Chen et al., 2023), so none of our stages is asked for one: the marginal and the cost head only order the search, and the search stays complete.

3. Problem statement and preliminaries

3.1. Instances and scope

We study feasible instances of (1): every instance is generated by planting a solution, so $\mathcal{S} \neq \emptyset$, and deciding infeasibility is outside the scope of this paper. The construction is the market-split recipe of Section 2 with 0/1 coefficients. The generator draws a dense random A (each entry is 1 with probability $\frac{1}{2}$), plants a random $\mathbf{x}^*$ with exactly $n/2$ ones and sets $\mathbf{b} = A\mathbf{x}^*$, so every right-hand side sits near the middle of its row's range, where the number of 0/1 completions is largest and propagation cannot fire. Among 20 candidate plantings per A it keeps the one whose relaxation polytope is widest, measured by the mean distance between the vertices returned by four random linear objectives; the relaxation of the kept instance is therefore as uninformative as the generator can make it. Uniqueness is not imposed, and m is chosen per size to match solution multiplicity (Section 5.1), because with $|\mathcal{S}| = 1$ every variable is FORCED and the task degenerates into predicting one vector.

3.2. Conditioning is reduction

A node of a depth-first search over (1) is a partial assignment $\mathbf{x}_F = \mathbf{v}$ on a set F of fixed variables, with K the free ones. Let $\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\}$ be the solutions consistent with the node and

$$p_j = \Pr[x_j{=}1 \mid A, \mathbf{b}, \mathbf{x}_F{=}\mathbf{v}] = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j{=}1\}|}{|\mathcal{S}(F, \mathbf{v})|}, \quad j \in K, \tag{2}$$

the conditional marginal, the posterior under the uniform prior on $\mathcal{S}$ that a planted-solution generator induces.

Proposition 1. *With A_F, A_K the column submatrices indexed by F, K, the map $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection from $\mathcal{S}(F, \mathbf{v})$ onto $\{\mathbf{y} \in \{0,1\}^{|K|} : A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}\}$. Hence the conditional marginals of the original instance are the ordinary marginals of the reduced instance.*

Proof. $A\mathbf{x} = A_F\mathbf{x}_F + A_K\mathbf{x}_K$; with $\mathbf{x}_F = \mathbf{v}$ fixed, $A\mathbf{x} = \mathbf{b}$ iff $A_K\mathbf{x}_K = \mathbf{b} - A_F\mathbf{v}$, and $\mathbf{x}_F$ is determined, so the map is a bijection; the marginal statement follows by counting. □

Two consequences (Figure 2). First, one architecture, one label generator and one training loop serve every node: a depth-d training state is stored as an instance on $n - d$ variables, and a network trained on small instances is evaluated on the reduced instances that a search on a larger one produces. Second, $p_j \in \{0, 1\}$ means every surviving solution agrees on x_j, and branching the other way empties the subtree. We call such variables FORCED and the rest OPEN: either branch of a OPEN variable still reaches a solution, so a wrong first value costs nothing beyond exploring a subtree that contains one. Only FORCED errors cost a backtrack.

3.3. What a node looks like, and the accuracy ceiling

Because $\mathcal{S}(F, \mathbf{v})$ can be enumerated on small instances, the Bayes-optimal per-variable accuracy of any predictor of a solution is computable,

$$\mathrm{ceil}(F, \mathbf{v}) = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \tag{3}$$

attained by answering the majority value of the surviving solutions. Figure 8a shows how a node changes with depth on 10×25 instances: the surviving solution set shrinks from a median of 21 at the root to a singleton by depth 7, and the FORCED share of the free variables rises from 7.5% to 61% at depth 5, 88% at depth 8 and 98% at depth 12. At the root almost no decision can cost anything and the ceiling (3) is dominated by OPEN variables; from depth 5 on, most decisions can. This is the regime a branching heuristic must be good in, and it is where prediction is trained and evaluated below.

4. Proposed method

4.1. Overview

The solver has two stages (Figure 1). Stage A is lattice reduction in the Aardal–Hurkens–Lenstra embedding, a sound classical method that returns a verified solution when it succeeds. Stage B is a depth-first search whose branching decisions are guided by a learned network; nothing learned ever fixes a variable or prunes, so the search stays complete. The network (Figure 4) is trained in two stages (Figure 5): Stage 1 fits its marginal head to exact conditional marginals of enumerated instances, Stage 2 adds a policy head on the frozen trunk and fine-tunes it on the exact number of nodes that every branching decision costs.

4.2. Lattice stage

The embedding of Aardal et al. (2000b) maps (1) to a lattice in $\mathbb{Z}^{n+1+m}$ with basis rows $(2\mathbf{e}_j \mid 0 \mid NA_{:,j})$ for $j = 1, \ldots, n$ and $(\mathbf{1} \mid 1 \mid N\mathbf{b})$, with N a large constant. For a solution $\mathbf{x}$, the integer combination with coefficients $(\mathbf{x}, -1)$ has constraint part $N(A\mathbf{x} - \mathbf{b}) = \mathbf{0}$, indicator -1 and variable part in $\{\pm 1\}^n$: a vector of squared length $n + 1$, among the shortest in the lattice because any vector that violates a row carries a multiple of N. We LLL-reduce the basis and then BKZ-reduce it with block size 12 (Lenstra et al., 1982; Schnorr and Euchner, 1994), scan the reduced basis for rows of that form, decode $x_j = (1 - w_j w_{n+1})/2$, verify $A\mathbf{x} = \mathbf{b}$ and return only verified solutions; up to ten random column permutations are tried. The stage costs 0.001 s per instance at $n = 25$, 0.16 s at $n = 60$ and 0.54 s at $n = 80$.

Its coverage falls with size for a reason that is visible in the reduced basis (Figure 3). Vectors with indicator 0 and zero constraint part correspond to integer kernel vectors $\mathbf{z}$ with $A\mathbf{z} = \mathbf{0}$; their variable part is $2\mathbf{z}$, so a kernel vector with k nonzero entries has squared length $4k$, and the kernel has rank $n - \mathrm{rank}\, A \approx 0.65\, n$ on our instances. At $n = 25$ the shortest kernel vector in the reduced basis has median squared length 16 against a solution's 26, and every basis contains a solution row.

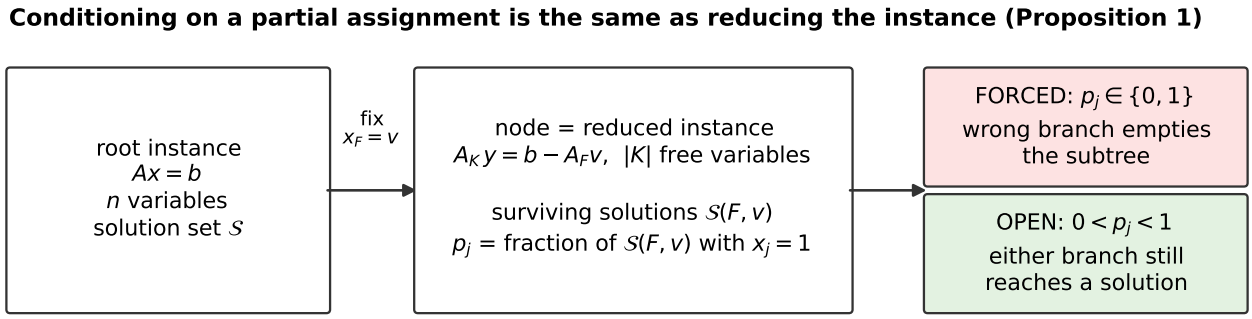


Figure 2: Proposition 1 and the partition it induces. A node with $\mathbf{x}_F = \mathbf{v}$ is the smaller BLS $A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}$; its conditional marginals are the ordinary marginals of that instance. Variables on which every surviving solution agrees are FORCED; only branching against them costs a backtrack.

At $n = 60$ the median shortest kernel vector has squared length 40 against 61 and 39 of the 61 basis rows are kernel vectors; a solution row survives in 2 of 30 bases under one permutation and in 33–53% of instances with ten. At $n \geq 70$ the shortest kernel vectors are shorter than the solution by 13–19 units and a solution row never appears. Reduction fills the basis with the shortest vectors it can find, and from $n \approx 60$ those are kernel vectors, not solutions. This is why the learned search, not the lattice, carries the sizes we care about.

4.3. Guided depth-first search

Every instance the lattice stage does not close is searched from scratch by the depth-first procedure of Algorithm 1. A node is a partial assignment; by Proposition 1 it is handled as the reduced instance $(A', \mathbf{b}') = (A_K, \mathbf{b} - A_F\mathbf{v})$, and r_i denotes the number of free variables in row i of A'.

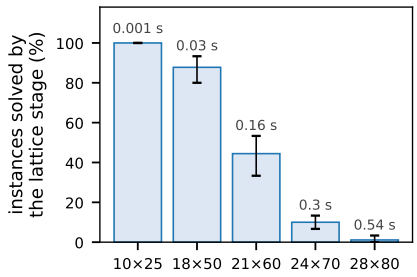
Three design choices matter for what follows. Propagation on a 0/1 row reads "exactly b'_i of the r_i free variables in this row are 1" and can fix variables only when $b'_i \in \{0, r_i\}$, which is why it is inert near the root of our instances. The relaxation is not rebuilt per node: one linear program over the original n variables is kept for the whole search and a node is expressed as a set of bound changes, so consecutive nodes, a child or a sibling after backtracking, differ by a few bounds and the dual simplex resumes from the parent basis; this is 50× cheaper than rebuilding and is what lets the loop process about 700 nodes per second in Python. Its infeasibility test is strictly stronger than propagation, since it sees linear combinations of rows, and it is the last sound step before the network. The vertex $\mathbf{x}^{LP}$ it returns is the main input feature of the network.

4.4. Stage 1: exact conditional marginals as the target

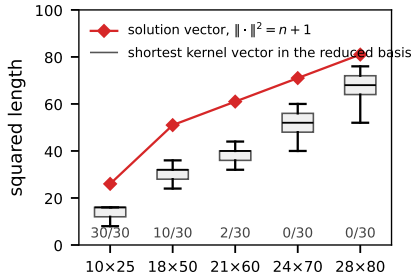
Network.. The network f_θ (Figure 4) is a bipartite variable–constraint graph network with graph attention (Veličković et al., 2018), the architecture family of Gasse et al. (2019). Each variable node carries three features (its relaxation value x_j^{LP}, its column density, its fractionality $|x_j^{LP} - \text{round}(x_j^{LP})|$), each constraint node three (its right-hand side b'_i/r_i, its density $r_i/|K|$, its relaxation residual $(b'_i - (A'\mathbf{x}^{LP})_i)/r_i$), and an edge joins j to i where $a'_{ij} = 1$. Linear layers lift both to width 64; four rounds of attention then alternate variable-to-constraint and constraint-to-variable messages with residual connections and layer normalisation, and a two-layer marginal head maps each variable embedding $\mathbf{h}_j$ to one logit, $\hat{p}_j = \sigma(m_j)$; 21,761 parameters in all. The two heads added in Stage 2 are described in Section 4.5.

Table 1: Algorithm 1: one node of the guided depth-first search. Steps 1–4 are sound and are the only steps that fix variables or prune; the learned components enter only at step 5.

1	**Reduce.** Pop $(F, \mathbf{v})$; form $A' = A_K$, $\mathbf{b}' = \mathbf{b} - A_F\mathbf{v}$. Drop rows with $r_i = 0$; if none remain, verify $A\mathbf{x} = \mathbf{b}$ and return $\mathbf{x}$.
2	**Bound check.** If $b'_i < 0$ or $b'_i > r_i$ for some row, discard the node.
3	**Propagate.** While some row has $b'_i = 0$ (fix its free variables to 0) or $b'_i = r_i$ (fix them to 1): apply the fixings; on a contradiction discard the node; if variables were fixed, push the reduced node and return.
4	**Relax.** Re-optimise the single warm-started relaxation $\{A\mathbf{x} = \mathbf{b},\ 0 \leq \mathbf{x} \leq 1\}$ with the bounds of F tightened to $\mathbf{v}$ (dual simplex from the previous basis, 0.08 ms). If infeasible, discard the node; else read the vertex $\mathbf{x}_K^{LP}$.
5	**Predict.** One forward pass of the network on the bipartite graph of $(A', \mathbf{b}', \mathbf{x}_K^{LP})$ gives $\hat{p}_j$ and g_j for $j \in K$ (1.9 ms at a 21×60 root). Choose $j^\star = \arg\max_j g_j$ and $v = \mathbb{1}[\hat{p}_{j^\star} \geq \frac{1}{2}]$.
6	**Branch.** Push the child $x_{j^\star} = 1 - v$, then the child $x_{j^\star} = v$, so that v is explored first and $1 - v$ on backtracking.



(a) coverage: 10 permutations, 3 seeds (range); time per instance above



(b) why it fails: kernel vectors become shorter than the solution
(numbers at the bottom: bases with a solution row, one permutation)

Figure 3: The lattice stage. (a) Fraction of instances it solves with ten column permutations, three seeds (bars: mean, whiskers: range; number above: median time per instance). (b) Squared length of the shortest kernel-type vector in the reduced basis (boxes over 30 instances, one permutation) against the squared length $n+1$ of a solution vector; the numbers at the top count the instances whose reduced basis contains a solution row.

The architecture is standard; the contribution is the target.

Target and training states.. On instances small enough to enumerate (10×25), $\mathcal{S}$ is obtained with the all-solutions mode of a constraint solver and p_j computed by counting. Only enumerations that run to exhaustion are accepted; a time-limited enumeration with solutions already collected would substitute a truncated $\mathcal{S}$ and bias every label. Training states are reachable in-tree states: a depth $d \in \{0, \dots, 12\}$ is drawn uniformly, a prefix of an LP-confidence variable ordering is fixed to the values of a randomly chosen actual solution, and the reduced instance is enumerated in turn to give p_j on its free variables. The network is trained by cross-entropy against the soft targets,

$$\mathcal{L}_1 = -\tfrac{1}{|K|} \sum_{j \in K} [p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j)], \qquad (4)$$

which is minimised exactly at $\hat{p} = p$; replacing p_j by the planted solution, the usual choice, is a noisy surrogate whenever $|\mathcal{S}| > 1$.

Branching rule.. At a node the network is evaluated once and the Stage-1 rule branches on the most confident marginal with its majority value,

$$j^\star = \arg\max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \qquad v = \mathbb{1}[\hat{p}_{j^\star} \geq \tfrac{1}{2}]. \qquad (5)$$

4.5. Stage 2: exact subtree node counts as the objective

Rule (5) maximises the chance that the first child contains a solution. It says nothing about how much the search pays when it does not, nor how much smaller the children become; both are properties of the subtree, and a depth-first search produces them exactly. For a node u let $c(u)$ be the number of nodes expanded below u until a solution is found there or the subtree is exhausted; the search returns it when it backtracks out of u. We use $c(u)$ as the cost of the branching decision made at u; $c(\text{root})$ is the number of nodes the whole search expands to reach a solution, which is the tree-size metric reported throughout the experiments. Stage 2 fine-tunes the branching order on $-\log c(u)$.

Decision process.. A state is the reduced instance after propagation; an action is a free variable j; the first value stays $\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$ from the frozen Stage-1 network. Because each decision owns its subtree, credit is assigned per decision rather than per episode, the property a sparse solved/unsolved reward lacks and the one that makes the problem well posed (Etheve et al., 2020).

Policy and value heads.. On the frozen trunk of f_θ two heads are added (Figure 4). The policy logit is

$$g_j(s) = \log\left|\hat{p}_j - \tfrac{1}{2}\right| + h_\phi(\mathbf{h}_j), \qquad (6)$$

with h_ϕ a two-layer network whose last layer is initialised to zero, so that $\pi(j \mid s) \propto \exp(g_j/\tau)$ starts as a softening of rule (5) and its argmax is exactly that rule. A value head $V_\psi(s)$ on the mean- and max-pooled variable and constraint embeddings, $\log|K|$ and $\log m$ predicts $\log c$.

Update.. Each epoch samples 400 training states with 45–60 free variables, rolls the sampled policy ($\tau = 1$) out to a solution, and records every decision with its exact cost, about 21,000 decisions per epoch. With the advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$ standardised within the epoch,

$$\mathcal{L}_2 = \sum_u \Big[-\alpha(u) \log \pi(j_u \mid s_u) - \lambda \mathcal{H}(\pi(\cdot \mid s_u)) + (V_\psi(s_u) - \log c(u))^2 \Big], \qquad \lambda = 0.01, \qquad (7)$$

an on-policy actor-critic update with Monte-Carlo returns (Williams, 1992); no bootstrapping is needed because the return is exact. Only ϕ, ψ are trained; θ is fixed, so the marginal that chooses values and defines the FORCED/OPEN partition is unchanged, and any improvement is attributable to the branching order alone. The critic is pre-trained on decisions recorded under rule (5), and its held-out rank correlation with $\log c$ gates the actor update, since a baseline that cannot rank costs turns the advantage into noise. The epoch is selected by the total node count below held-out states of the training pool, never on evaluation instances. At deployment the policy is greedy, $j^\star = \arg\max_j g_j(s)$.

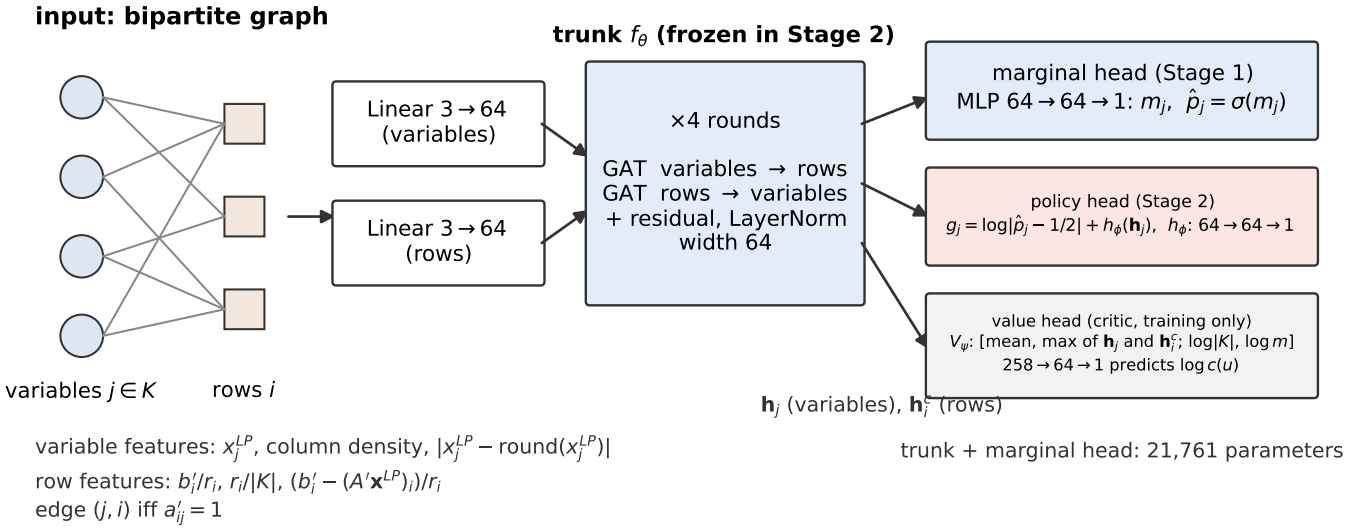


Figure 4: The network. The reduced instance enters as a bipartite graph with three features per variable and per row; a trunk of four graph-attention rounds produces an embedding per variable and per row; three heads read them. The marginal head is trained in Stage 1, the policy head in Stage 2 on the frozen trunk, and the value head serves only as the critic during Stage 2.

4.6. Cost per node

At a node with $|K|$ free variables the guided search pays one warm-started relaxation and one forward pass; the forward pass is over 90% of the node cost at 21×60. The sound procedure that detects the same FORCED variables, LP-probing (fix x_j to each value and prune the one that makes the relaxation infeasible), pays $|K|$ relaxations; with warm starts on both sides the measured ratio grows from 1.7× at $n = 25$ to 7.2× at $n = 60$. The Stage-2 heads add nothing measurable.

Table 2: Branching arms. All run in the same depth-first loop (Algorithm 1: one warm-started relaxation per node, the same propagation, the same budget) and differ only in step 5; the number in parentheses is the size the learned component was trained at. The LP rule is not a solver: it is the same loop with the relaxation vertex in place of the network.

arm	branches on	first value
random	a random free variable	random
LP rule	most integral relaxation value, $\arg\max_j \lvert x_j^{LP} - \frac{1}{2}\rvert$	round(x_j^{LP})
Stage 1 (25)	most confident marginal, $\arg\max_j \lvert\hat{p}_j - \frac{1}{2}\rvert$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$
Stage 1+2 (60 / 50)	cost-tuned score, $\arg\max_j g_j$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$

5. Experiments

The experiments answer four questions: does learned branching beat classical branching in the same search (Section 5.2); does it transfer to sizes it was not trained at (Section 5.3); how far is it from a complete solver (Section 5.4); and is every component necessary (Section 5.5).

5.1. Setup

Instances.. Instances are generated as in Section 3.1, the market-split construction with 0/1 coefficients and a planted solution. Sizes are 10×25, 18×50, 21×60, 24×70 and 28×80; m is chosen per size so that the median number of solutions is comparable where it can be verified by enumeration ($|\mathcal{S}|$: 23, 19 and 10 at $n = 25$, 50, 60; 27/30 verified at 60) and by constraint density $m/n \approx 0.35$ at $n \geq 70$, where enumeration does not complete. At $n = 100$ CP-SAT finds no solution within 20 s and every strategy fails, so $n = 80$ is the last size. Evaluation uses 30 instances per size and a 100-instance 21×60 set (generator seeds 700000–700099, which contains the thirty); Stage 1 trains on 10,000 in-tree states from 1,760 separate 10×25 instances, Stage 2 on a separate 21×60 pool of 500 instances (seeds 800000+, 400/100 split) whose 100 held-out instances supply the epoch-selection states. Overlap between any training pool and any evaluation set is zero by hash of $(A, \mathbf{b})$.

Training.. Stage 1: Adam, learning rate 10^{-3}, 20 epochs, batch 1, gradient clipping 1.0; 20 minutes on one CPU core; no hyperparameter was tuned and there is no validation split. Stage 2: critic pre-training on 37,275 recorded decisions (9,375 held-out), then six epochs of the update (7); two training seeds. The policy fine-tuned at 21×60 is denoted Stage 1+2 (60); a second policy fine-tuned at 18×50 with the same protocol, Stage 1+2 (50), tests upward transfer.

Arms and solvers.. Table 2 defines the branching arms. The random arm checks that the instances are hard; the LP rule is the classical most-integral rule and the reference for branching quality. CP-SAT (OR-Tools) and SCIP are run as complete solvers on the same instances, single-threaded, under the same budgets, as the wall-clock reference.

Protocol and metrics.. Two metrics are reported per instance: the number of nodes the search expands to reach a solution, c(root) of Section 4.5, which measures branching quality and is

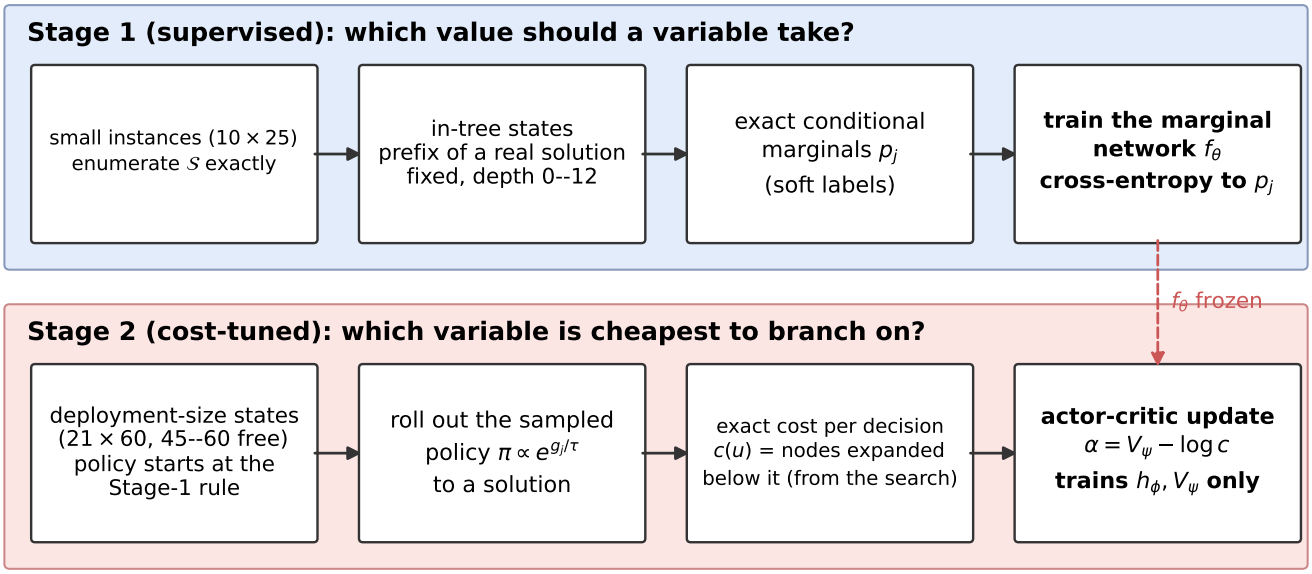


Figure 5: The two training stages. Stage 1 learns *which value* from exact conditional marginals of enumerable instances; Stage 2 learns *which variable* from the exact number of nodes every decision costs, produced by the search, with the Stage-1 network frozen.

Table 3: Branching strategies in the same warm-started loop, lattice stage off, three seeds (mean ± s.d. of per-seed medians; node counts are seed-invariant). Stage 1+2 is the policy fine-tuned at 21 × 60, reported for two training seeds.

size (budget)	strategy	solved	median nodes	median time
10 × 25 (60 s)	LP rule	30/30	46	0.03 s
	Stage 1	30/30	24	0.07 s
18 × 50 (300 s)	LP rule	30/30	4,224	2.43 ± 0.33 s
	Stage 1	30/30	1,085	1.82 ± 0.37 s
	Stage 1+2	30/30	**424**	**0.52 ± 0.01 s**
21 × 60, 100 inst. (600 s)	LP rule	100/100	79,609	41.2 ± 1.9 s
	Stage 1	100/100	13,386	16.5 ± 0.6 s
	Stage 1+2 (seed 0 / 1)	100/100	5,495 / **4,940**	7.6 / **6.8** s

Table 4: Instances solved within budget by size, lattice off; 30 instances per size (100 at 21 × 60), ranges over three seeds where they differ, 28 × 80 one seed. Training sizes in parentheses.

size (budget)	LP rule	Stage 1 (25)	Stage 1+2 (60)	Stage 1+2 (50)
18 × 50 (300 s)	30/30	30/30	30/30	30/30
21 × 60, 100 inst. (600 s)	100/100	100/100	100/100	100/100
24 × 70 (1200 s)	23–25/30	29/30	**30/30**	**30/30**
28 × 80 (1200 s)	4/30	8/30	**18/30**	17/30

Table 5: Median nodes to a solution by size, lattice off (seed-invariant; same instances as Table 4, medians over solved instances at 28 × 80).

size	LP rule	Stage 1 (25)	Stage 1+2 (60)	Stage 1+2 (50)
18 × 50	4,224	1,085	**424**	1,147
21 × 60, 100 inst.	79,609	13,386	**5,495**	7,796
24 × 70	980,743	260,114	**90,054**	95,818
28 × 80	1,929,041	803,535	**470,606**	560,077

seed-invariant because the search is deterministic given the relaxation vertex; and wall-clock time, which also reflects the cost per node. Budgets are per-instance wall-clock (60, 300, 600, 1,200, 1,200 s for the five sizes), enforced by process termination; six instances run concurrently on a 20-core machine and all timing-sensitive code is single-threaded (no GPU is used). Per-instance speed is compared with a two-sided sign test. Every 21 × 60 figure is replicated over three evaluation seeds, and the lattice stage is switched off for the branching comparisons and on for the cascade (Section 5.5.4). Enumeration and CP-SAT use OR-Tools; lattice reduction uses fpylll; the relaxation uses Gurobi's dual simplex.

5.2. *Learned branching against classical branching*

Table 3 and Figure 6b give the main result. Stage 1 produces a smaller tree than the LP rule at every size, by 2.0×, 3.9× and 5.5× nodes, and the ratio grows with size. A smaller tree buys time only once the tree is large enough to pay for the forward pass: at 10 × 25 Stage 1 is slower, from 18 × 50 it is faster (1.3–1.4×, not significant on 30 instances), and on the 100-instance 21 × 60 set it is 2.0–2.3× faster on 70–71 of 100 instances ($p < 0.001$). Stage 2 removes a further 2.3–2.4× of the nodes and 2.0–2.4× of the time (64–68/100, $p \leq 0.007$) at unchanged per-node cost, in both training seeds and every evaluation seed, for 4.8–5.9× over the LP rule end to end; the fine-tuned policy also transfers down to 18 × 50 (2.6× fewer nodes, 1.8–2.2× faster, $p \leq 0.016$). Random branching solves none of 30 instances at 20 × 50 within 300 s and none of the 21 × 60 residual of Section 5.5.4: the instances are hard, and every sound arm closes all of them, so the comparison is about nodes and time.

5.3. *Transfer across sizes*

Both stages transfer in both directions (Tables 4–6, Figure 7). Stage 1, trained only at 10 × 25, keeps a node advantage over

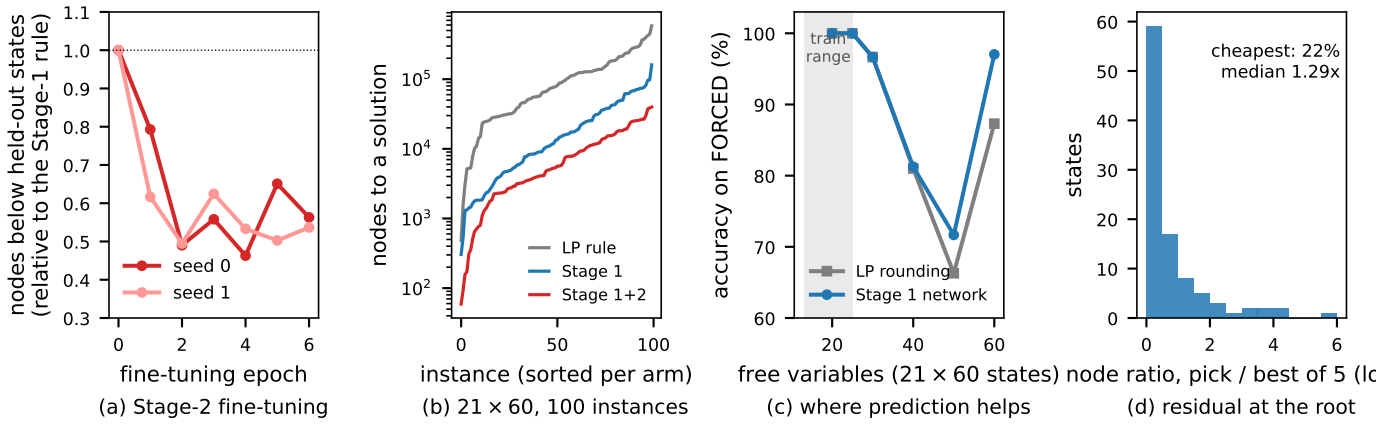


Figure 6: (a) Stage-2 fine-tuning: nodes expanded below 150 held-out states relative to the Stage-1 rule, two training seeds; the selected epochs are 4 and 2. (b) Nodes to a solution on the 100-instance 21 × 60 set, each arm sorted. (c) Accuracy on FORCED variables of 21 × 60 in-tree states by number of free variables: the Stage-1 margin over relaxation rounding lies outside its training range. (d) Residual at the root: nodes below the confidence rule's choice relative to the cheapest of its five candidates (100 held-out states, 45–50 free variables).

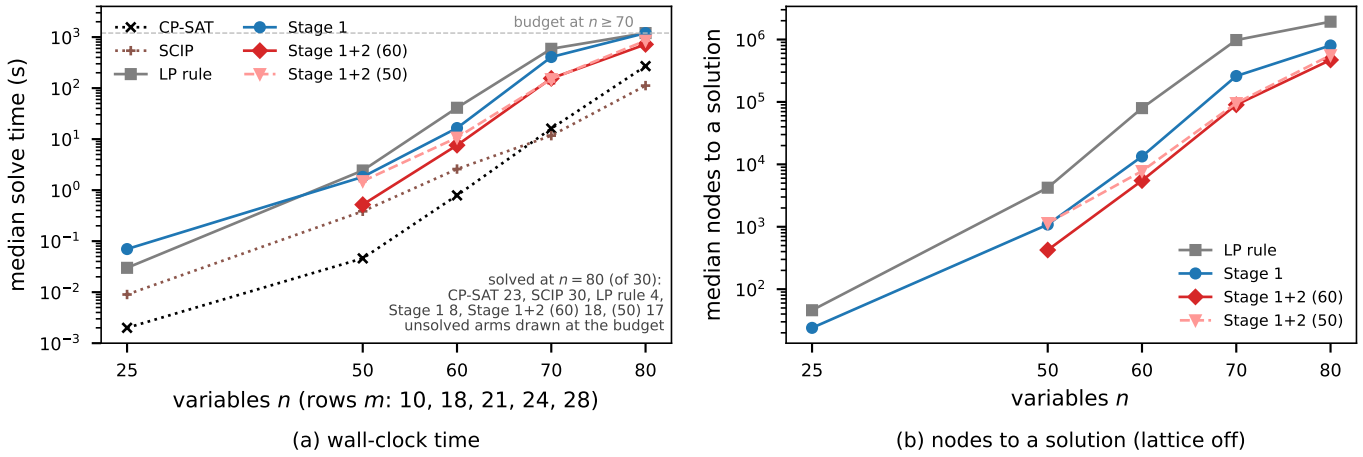


Figure 7: Scaling with size. (a) Median wall-clock time of the branching arms and of the complete solvers; at $n \geq 70$ whiskers span the three seeds, and unsolved arms at $n = 80$ are drawn at the budget with their solve counts in the panel. (b) Median nodes to a solution, lattice off. Both stages were trained at $n \leq 60$.

Table 6: The same results read by transfer direction: ratio of median nodes, range of per-instance median time ratios over seeds, and the number of instances on which the transferred arm is faster.

trained → tested	vs.	nodes	time	wins
Stage 1: 25 → 50	LP rule	3.9×	1.3–1.4×	18/30
Stage 1: 25 → 60	LP rule	5.5×	2.0–2.3×	70–71/100
Stage 1: 25 → 70	LP rule	3.8×	0.7–1.1×	16–17/30
Stage 1+2: 60 → 50 (down)	Stage 1	2.6×	1.8–2.2×	22–24/30
Stage 1+2: 50 → 60 (up)	Stage 1	2.0×	1.7–1.9×	63–66/100
Stage 1+2: 60 → 70 (up)	Stage 1	2.7×	2.3–3.3×	22–23/30
Stage 1+2: 50 → 70 (up)	Stage 1	2.2×	2.1–2.7×	19–21/30
Stage 1+2: 50 → 50 (same)	Stage 1	0.9×	1.1–1.4×	16–18/30

the LP rule at every size (3.9×, 5.5×, 3.8×, 2.4× at $n = 50$, 60, 70, 80), but its time advantage shrinks as the tree grows: at 24 × 70 it solves 29/30 against the LP rule's 23–25/30, yet on the instances both solve it is no faster (0.7–1.1×), because a 260,000-node tree pays for 260,000 forward passes. Stage 2 restores the advantage and carries it upward. The policy fine-tuned at 18 × 50 is 2.0× better than Stage 1 in nodes at 21 × 60 (63–66/100 faster, $p < 0.001$); both fine-tuned policies solve 30/30 at 24 × 70 in every seed, 2.3–3.3× and 2.1–2.7× faster than Stage 1, at 1.2–1.4× their training size; at 28 × 80 they solve 18 and 17 of 30 within 1,200 s where Stage 1 solves 8 and the LP rule 4. That is where the window closes for the learned search: SCIP solves 30/30 there at a median of 112 s.

5.4. *Distance to complete solvers*

Table 7 places the learned search next to the complete solvers on the same instances. CP-SAT is 9–11× faster than the full method at 18 × 50 and 21 × 60, and from 24 × 70 SCIP is the fastest, 10–15× ahead. Table 8 decomposes the gap: CP-SAT does not search a smaller tree than we do, it processes each node about 28× faster, in C++ with clause learning and restarts, where our loop runs in Python and spends over 90%

Table 7: Median wall-clock time by size, single-threaded, same instances and budgets; where an arm does not solve every instance its solve count is in Table 4 (CP-SAT 23/30 at 28 × 80).

size	CP-SAT	SCIP	LP rule	Stage 1	Stage 1+2 (60)	Stage 1+2 (50)
10 × 25	**0.002 s**	0.009 s	0.03 s	0.07 s	—	—
18 × 50	**0.05 s**	0.39 s	2.4 s	1.8 s	0.5 s	1.5 s
21 × 60, 100 inst.	**0.8 s**	2.6 s	41 s	16.5 s	7.6 s	10.7 s
24 × 70	16 s	**12 s**	537–644 s	358–457 s	125–185 s	135–165 s
28 × 80	270 s	**112 s**	>1200 s	>1200 s	720 s	852 s

Table 8: Decomposition of the gap to CP-SAT at 21 × 60 (30 instances). CP-SAT's branch counter and our node counter are not the same unit (branches include restarts and exclude what clause learning skips), so the first row is a comparison of magnitude only.

	CP-SAT	Stage 1+2
branches / nodes to a solution (median)	18,827	4,040–8,873
branches / nodes per second	19,782	≈ 700
median time	1.16 s	5.9–12.5 s

of each node in the network's forward pass. The deficit is per-node throughput, not branching quality; whether a compiled inference path and clause learning close it is not shown here.

5.5. Ablations

5.5.1. The Stage-1 signal: in-tree labels, then graph structure

Table 9 and Figure 8 isolate what makes the Stage-1 signal work. The in-tree labels come first. A network with the same architecture, labels, optimiser and number of states, trained on root states only, is better than the in-tree network at the root (67.9% against 66.7% on all free variables) and worse from depth 4 on; over all depths it is below relaxation rounding on every variable (75.5% against 76.5%) and, on the FORCED variables at depth 8, 3.4 points below it. Root-level accuracy is the wrong objective for a branching heuristic because almost nothing at the root can cost anything. The graph structure comes second. A graph-free MLP on the same per-variable features plus mean and max aggregates of its incident rows, trained on the same in-tree states, reaches 89.5% on FORCED variables against the GAT's 93.2%; on a matched 2,000/800 split its L_1 distance to the exact marginal is 0.225 against 0.151. The label effect (5–7 points on FORCED variables) is larger than the structure effect (3–4 points), and both are needed to stay above the classical rule at every depth. Within the training size, the depth range of the training states does not matter (depths 0–6, 6–12 and 0–12 give 79.3, 79.9 and 79.9%).

Where the margin lives at deployment size is shown in Figure 6c on 21×60 in-tree states: inside the training range (13–25 free variables) the network and rounding are indistinguishable, because at constraint density $m/|K| \geq 0.8$ the relaxation is already integral on every FORCED variable; the entire margin (+5 points) arises at 50–60 free variables, outside the training range, where the relaxation is weakest. 80% of the queries a 21 × 60 search makes fall there.

5.5.2. The two signals are not interchangeable

Sharpening the value signal enlarges the tree.. Figure 6c suggests an obvious fix: train Stage 1 where its margin lives. We

Table 9: Accuracy on FORCED variables of held-out in-tree states of 10 × 25 instances, where an error costs a backtrack; ground truth is the reference solution, and the ceiling is 100% by definition. Accuracy on all free variables is in Figure 8b.

predictor	all depths	depth 8
GAT, in-tree (Stage 1)	**93.2%**	**89.4%**
MLP, in-tree	89.5%	86.2%
GAT, root-only (control)	88.0%	81.9%
LP rounding	87.9%	85.3%
FORCED share of variables	57.2%	87.9%

built 2,691 in-tree states with 30–60 free variables from the 21 × 60 training pool (exact marginals; every state with at most 55 free variables enumerates within 90 s) and trained the same architecture on them alone (M1) and on a half-and-half mix with the 10×25 states (M2). Accuracy on FORCED variables at 50 and 60 free variables rises by about one point (71.7 → 72.4% and 97.0 → 98.0%, Figure 9a). The tree grows: M1 expands a median of 21,337 nodes on the 100-instance set against 13,386 for Stage 1 (fewer on only 38/100, $p = 0.021$), M2 18,558, and a switch that routes nodes with at least 50 free variables to M1 and the rest to Stage 1 recovers nothing (17,252; Figure 9b). The reason is where the decisions are: two thirds of them occur at 25–34 free variables, and on such small dense states M1's most confident variable is FORCED-and-correct 77.6% of the time against 94.4% for Stage 1 (rank correlation with the true margin 0.12 against 0.36). A network slightly better at the top of the tree and much worse inside it makes the search slower; the training distribution of the value signal is not the lever.

The confident choice is rarely the cheapest.. On 100 held-out states with 45–50 free variables we forced the root to each of the five most confident variables and ran rule (5) below (Figure 6d). The rule's own choice is the cheapest of the five on 22% of states; the median ratio of its node count to the best candidate's is 1.29, the mean 2.97, a candidate at least twice as cheap exists on 24% of states, and the best root choice alone would remove 55% of the nodes summed over these states. The residual is about *which* variable, and it is large.

The cost signal moves away from confidence.. The critic on the frozen trunk reaches a held-out Spearman correlation of 0.80 with $\log c$ (a baseline that knows only the state's size: 0.70; within the 35–44 free-variable bucket 0.68 against 0.38); unfreezing the trunk adds 0.02 and is not used. Six fine-tuning epochs bring the number of nodes below held-out states to 0.46 (seed 0, epoch 4 selected) and 0.50 (seed 1, epoch 2) of the Stage-1 rule's, with the policy entropy falling from 1.7 to below 0.1 (Figure 6a). On 300 held-out states the fine-tuned policy branches on a different root variable than rule (5) at 95% of them; its pick is typically the 15th most confident ($|\hat{p} - \frac{1}{2}|$ of 0.26 against 0.43) and is FORCED with a correct first value less often (57.5% against 82.5%; Figure 9c). By the value criterion the fine-tuned choice is worse on every count, and the tree is less than half the size. The gain is not a property of the root

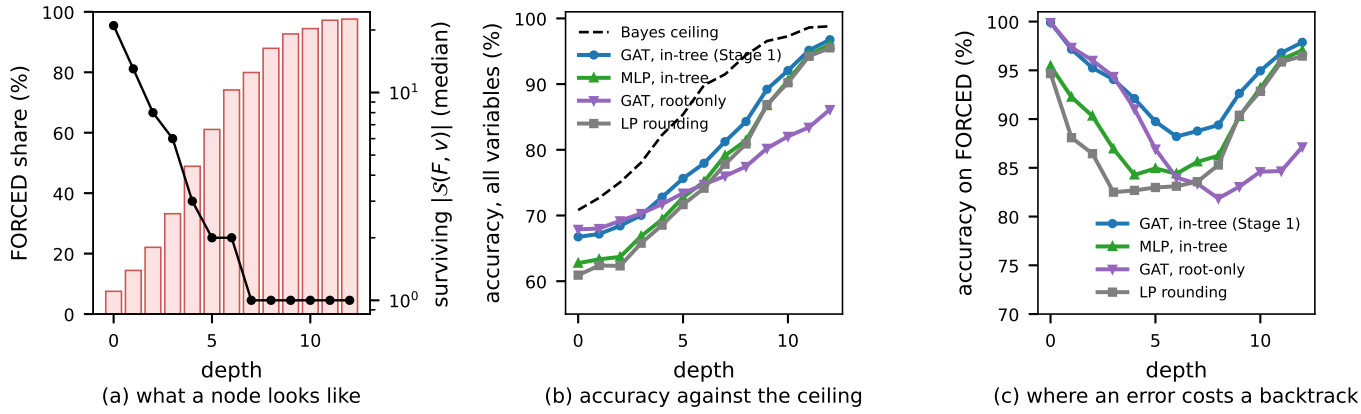


Figure 8: Prediction against the exact posterior on 2,640 held-out in-tree states of 10×25 instances, by depth. (a) The FORCED share of free variables and the median number of surviving solutions. (b) Per-variable accuracy on all free variables against the Bayes ceiling (3). (c) Accuracy on FORCED variables, where an error costs a backtrack. Four predictors share the labels, the loss and the states: the Stage-1 network (a GAT trained in-tree), the same GAT trained on root states only, a graph-free MLP with constraint aggregates trained in-tree, and relaxation rounding.

Table 10: Gain of the cost-tuned policy over the Stage-1 rule by evaluation size: ratio of median nodes and range of per-instance median time ratios over seeds.

condition	nodes	time
18×50, trained at 18×50 (tree of 10^3 nodes)	0.9×	1.1–1.4×
18×50, trained at 21×60	2.6×	1.8–2.2×
21×60, trained at 21×60	2.3–2.4×	2.0–2.4×
24×70, trained at 21×60	2.7×	2.3–3.3×
28×80, trained at 21×60	solved 8/30 → 18/30	

decision alone: forcing only the root to the fine-tuned pick with rule (5) below gives a median node ratio of 0.93 (cheaper on 22/46), so it is the sequence of decisions that improves. Two training seeds reach similar node counts while choosing different variables at many nodes, as expected if the objective, not a particular solution, was learned. The probability that a branch is right and the cost of it being wrong are different objectives, and the search minimises the second.

5.5.3. *Where Stage 2 matters*

The fine-tuned policy gains nothing at its own training size when the tree is small (18×50: 1,147 against 1,085 nodes), and its gain grows with the tree (Table 10). This is consistent with Figure 6c: what is learned matters on large, weakly constrained states, and those exist only in large trees. Below 18×50 the Stage-1 rule is sufficient; above 21×60 the cost signal is what keeps the learned search ahead of the LP rule at all.

5.5.4. *The lattice stage and the residual*

Tables 11 and 12 place the learned search where it is deployed. The lattice stage closes everything at $n = 25$, most instances at $n = 50$, a third to a half at $n = 60$ and almost nothing above; its failed attempts cost at most 0.7 s per instance, negligible against the search that follows. The 21×60 residual is genuinely hard, since random branching closes none of it, every sound strategy closes all of it, and the difference is time: Stage 1 is 3.0× faster than the LP rule on the residual (40/50, $p = 2 \times 10^{-5}$) and Stage 2 a further 1.2–2.7× depending on the seed's residual. At $n \geq 70$ the residual is the whole set and Tables 4 and 7 apply: at 24×70 the fine-tuned policies close 30/30 where the lattice closes 2–4 and the LP rule 23–25.

Table 11: Coverage of the lattice stage by size: instances it solves in each of three seeds (the seed fixes the column permutations) and its median time per instance, including failed attempts.

size	solved by the lattice stage	per instance
10×25	30, 30, 30 of 30	0.001 s
18×50	27, 28, 24 of 30	0.03 s
21×60 (30)	16, 14, 10 of 30	0.16 s
21×60 (100)	49, 38, 45 of 100	0.16 s
24×70	3, 4, 2 of 30	0.30 s
28×80	0, 1, 0 of 30	0.54 s

Table 12: The 21×60 residual left by the lattice stage (50 instances pooled over the three seeds; every arm within a seed receives the identical residual): instances solved within 600 s and the median time on them, range over seeds.

arm on the residual	solved	median time
random	0/50	> 600 s
LP rule	50/50	52.9–65.1 s
Stage 1	50/50	14.2–20.2 s
Stage 1+2	50/50	**7.3–11.7** s

6. Conclusion

A branching decision asks two questions, and this paper trains on an exact answer to each. The exact conditional posterior, computable where solutions can be enumerated and transported to every node by the reduction identity, tells the search which value to try and identifies which predictions can cost anything; the exact number of nodes below a decision, produced by the search as it backtracks, tells it which variable to branch on.

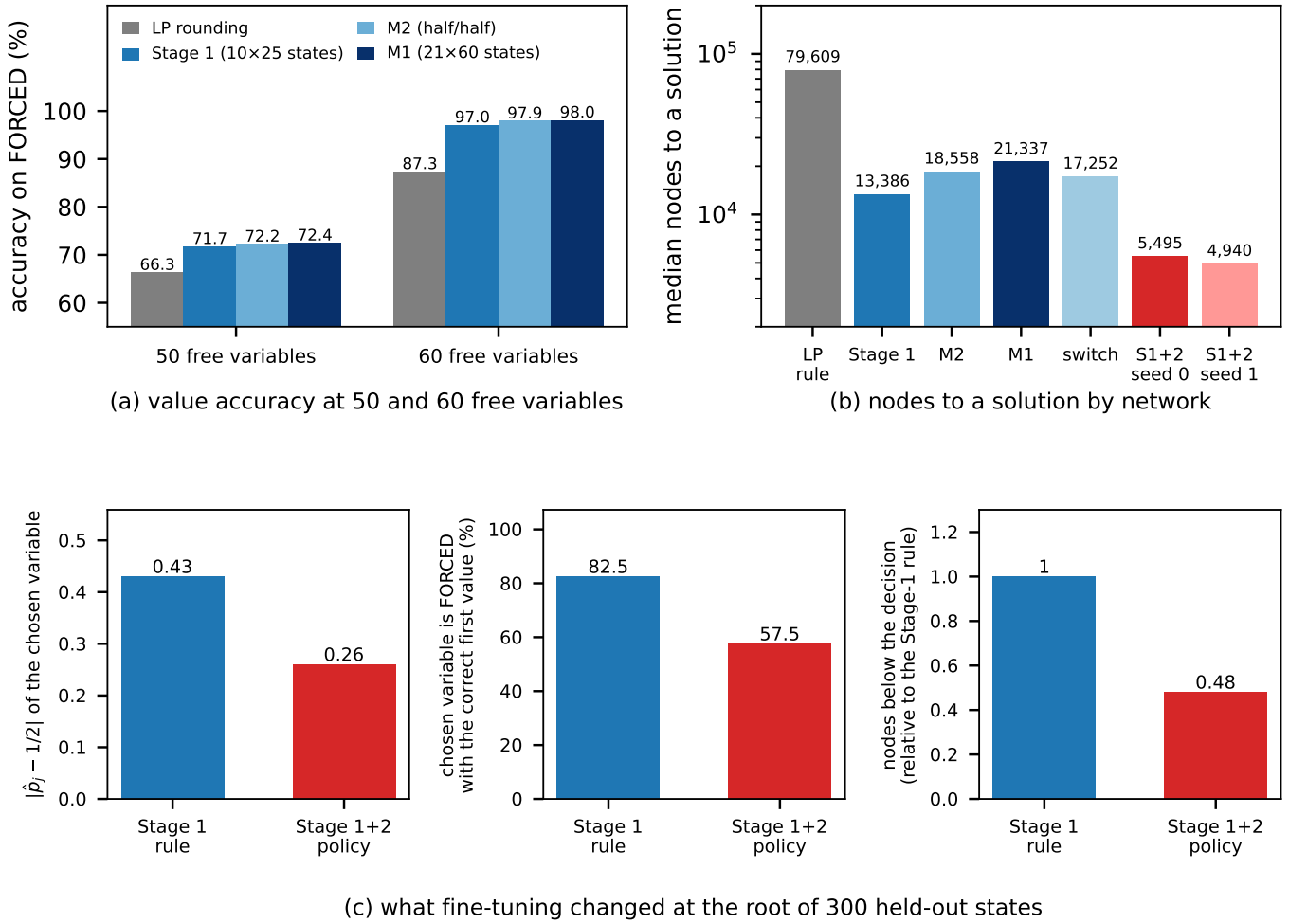


Figure 9: Value accuracy and tree size move independently. (a) Accuracy on FORCED variables of 21×60 in-tree states with 50 and 60 free variables, for relaxation rounding, the Stage-1 network trained on 10×25 states, and the same architecture trained on 21×60 states (M1) or on a half-and-half mix (M2). (b) Median nodes to a solution on the 100-instance set: the networks with the higher value accuracy expand more nodes; the cost-tuned policy expands fewer. (c) At the root of 300 held-out states the fine-tuned policy picks a less confident variable that is less often FORCED-and-correct, and the number of nodes below its decision is half.

Deployed behind a lattice stage on hard feasible binary linear systems, the first signal beats LP-guided branching by 2.0–2.3× on 100 instances of size 21×60; the second, applied as a fine-tuning that leaves the first untouched, adds 2.0–2.4×, replicates across seeds, and transfers upward to 24×70 and 28×80, where the first signal alone no longer beats the classical rule. That the fine-tuned policy prefers variables the posterior is less sure about, and that sharpening the posterior where its margin lives makes the tree larger, are the same fact seen twice: the probability of being right and the cost of being wrong are different objectives, and a search minimises the second.

Two limitations bound the result. Exact supervision bounds Stage 1: the marginals require enumerating $\mathcal{S}$, which does not complete beyond $n \approx 60$, so Stage 1 transfers to larger sizes but cannot be trained there, and the useful window of the learned search closes at $n \approx 80$. And the system is not competitive with complete solvers in wall-clock time: CP-SAT is 9–11× faster at $n \leq 60$ and SCIP 10–15× faster at $n \geq 70$, a deficit of per-node throughput in a Python loop dominated by the forward pass rather than of tree size.

Both point to the next steps. The cost signal of Stage 2 needs no enumeration, so the branching order can be fine-tuned at the sizes where enumeration fails, and, because the number of nodes below a decision is defined whether or not the subtree contains a solution, on infeasible instances, which this paper excludes. The throughput gap calls for a compiled inference path and for the clause learning and restarts that complete solvers use and our search lacks. Two ablations remain to be run, the cost-tuned variable choice combined with the LP rounding value and the search loop without its propagation or relaxation steps, and whether either signal transfers to other constraint families is untested.

References

Aardal, K., Bixby, R.E., Hurkens, C.A.J., Lenstra, A.K., Smeltink, J.W., 2000a. Market split and basis reduction: To-

wards a solution of the Cornuéjols–Dawande instances. INFORMS Journal on Computing 12, 192–202.

Aardal, K., Hurkens, C.A.J., Lenstra, A.K., 2000b. Solving a system of linear Diophantine equations with lower and upper bounds on the variables. Mathematics of Operations Research 25, 427–442.

Amos, B., 2023. Tutorial on amortized optimization. Foundations and Trends in Machine Learning 16, 592–732.

Cappart, Q., Moisan, T., Rousseau, L.M., Prémont-Schwarz, I., Cire, A.A., 2021. Combining reinforcement learning and constraint programming for combinatorial optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Chen, Z., Liu, J., Wang, X., Lu, J., Yin, W., 2023. On representing mixed-integer linear programs by graph neural networks, in: International Conference on Learning Representations.

Chu, G., Stuckey, P.J., 2015. Learning value heuristics for constraint programming, in: Integration of AI and OR Techniques in Constraint Programming (CPAIOR).

Cornuéjols, G., Dawande, M., 1999. A class of hard small 0-1 programs, in: Integer Programming and Combinatorial Optimization (IPCO), pp. 284–293.

Etheve, M., Alès, Z., Bissuel, C., Juan, O., Kedad-Sidhoum, S., 2020. Reinforcement learning for variable selection in a branch and bound algorithm, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR), pp. 176–185.

Feng, S., Yang, Y., 2025. SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A., 2019. Exact combinatorial optimization with graph convolutional neural networks, in: Advances in Neural Information Processing Systems.

Gershman, S.J., Goodman, N.D., 2014. Amortized inference in probabilistic reasoning, in: Proceedings of the Annual Meeting of the Cognitive Science Society.

Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., Luo, X., 2023. A GNN-guided predict-and-search framework for mixed-integer linear programming, in: International Conference on Learning Representations.

Haralick, R.M., Elliott, G.L., 1980. Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence 14, 263–313.

Khalil, E.B., Le Bodic, P., Song, L., Nemhauser, G., Dilkina, B., 2016. Learning to branch in mixed integer programming, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Khalil, E.B., Morris, C., Lodi, A., 2022. MIP-GNN: A data-driven framework for guiding combinatorial solvers, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Lenstra, A.K., Lenstra, H.W., Lovász, L., 1982. Factoring polynomials with rational coefficients. Mathematische Annalen 261, 515–534.

Li, Z., Guo, J., Si, X., 2023. G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. Transactions on Machine Learning Research .

Li, Z., Si, X., 2022. NSNet: A general neural probabilistic framework for satisfiability problems, in: Advances in Neural Information Processing Systems.

Marques-Silva, J.a.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521.

Montanari, A., Ricci-Tersenghi, F., Semerjian, G., 2007. Solving constraint satisfaction problems through belief propagation-guided decimation, in: Allerton Conference on Communication, Control, and Computing.

Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., Addanki, R., Hapuarachchi, T., Keck, T., Keeling, J., Kohli, P., Ktena, I., Li, Y., Vinyals, O., Zwols, Y., 2020. Solving mixed integer programs using neural networks. ArXiv:2012.13349.

Ohrimenko, O., Stuckey, P.J., Codish, M., 2009. Propagation via lazy clause generation. Constraints 14, 357–391.

Parsonson, C.W.F., Laterre, A., Barrett, T.D., 2023. Reinforcement learning for branch-and-bound optimisation using retrospective trajectories, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Pesant, G., Quimper, C.G., Zanarini, A., 2012. Counting-based search: Branching heuristics for constraint satisfaction problems. Journal of Artificial Intelligence Research 43, 173–210.

Refalo, P., 2004. Impact-based search strategies for constraint programming, in: Principles and Practice of Constraint Programming (CP), pp. 557–571.

Scavuzzo, L., Chen, F.Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., Aardal, K., 2022. Learning to branch with tree MDPs, in: Advances in Neural Information Processing Systems.

Schnorr, C.P., Euchner, M., 1994. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Mathematical Programming 66, 181–199.

Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., Dill, D.L., 2019. Learning a SAT solver from single-bit supervision, in: International Conference on Learning Representations.

Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R.B., Seshia, S.A., Bacchus, F., 2021. Learning branching heuristics for propositional model counting, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: International Conference on Learning Representations.

Wassermann, A., 2025. Solving the market split problem with lattice enumeration. ArXiv:2508.08702; to appear in Mathematical Programming Computation.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256.